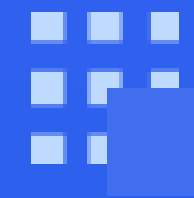




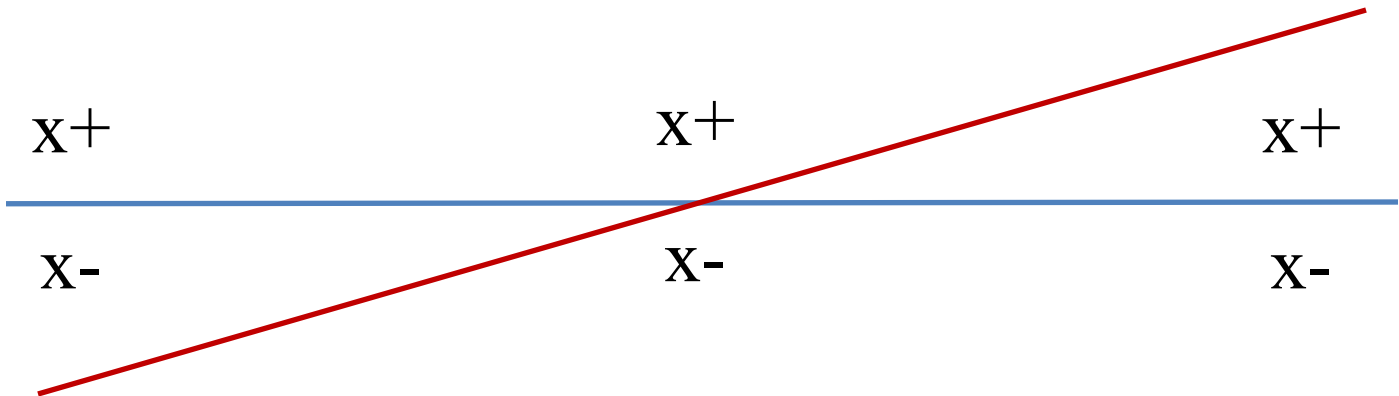
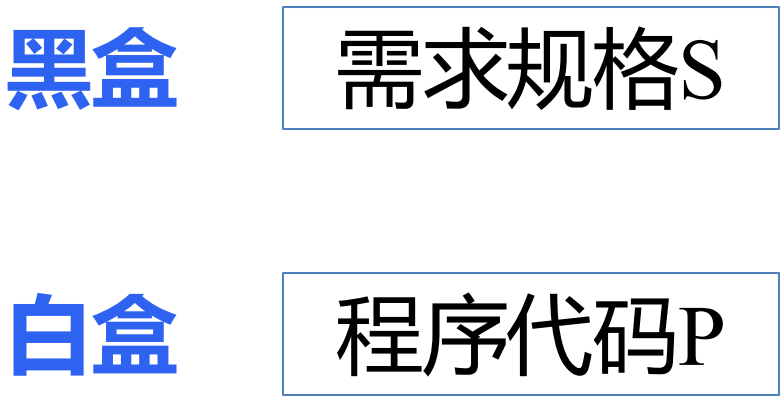
软件测试

白盒边界故障假设



白盒边界故障假设概述

- 蓝色需求规格S的分界线
- 红色程序实现P的分界线



分界点：以S还是以P作为基线设计故障假设？

白盒边界故障假设概述

黑盒

需求规格S

白盒

程序代码P

- 人类视角：代码
- 机器视角：计算

- 白盒边界故障假设主要关注代码中的判定决策点，如条件语句、循环语句，也包括更加细致的逻辑操作符。
- 通常包括输入范围的最小值、最大值，以及刚好超出范围的值。这里的值往往是逻辑的真假值。
- 深入理解代码逻辑，确定决策点和边界条件，设计和生成测试来验证系统是否能正确处理这些边界值。

边界故障假设 ■ 白盒边界故障测试

Triangle 程序的白盒边界故障假设

代码

```
1 def triangle(a, b, c):
2     if not (1 <= a <= 100 and 1 <= b <= 100 and 1 <= c <= 100):
3         print("无效输入值")
4     elif not (a + b > c and b + c > a and c + a > b):
5         print("无效三角形")
6     elif a == b and b == c:
7         print("等边三角形")
8     elif a == b or b == c or c == a:
9         print("等腰三角形")
10    else:
11        print("普通三角形")
```

- **第 2 行：**判定条件 ($1 \leq a \leq 100$ and $1 \leq b \leq 100$ and $1 \leq c \leq 100$) 的真假边界。
- **第 4 行：**判定条件 ($a + b > c$ and $b + c > a$ and $c + a > b$) 的真假边界。
- **第 6 行和第 8 行：**与“等边三角形”和“等腰三角形”的黑盒边界故障假设相似。

讨论与思考： Triangle程序中白盒故障假设和黑盒故障假设的相似与区别？

边界故障假设 ■ 白盒边界故障测试

MeanVar 程序的白盒边界故障假设

代码

```
1 #include <stdio.h>
2 void MeanVar(double X[], int L) {
3     double var, mean, sum, varsum;
4     sum = 0;
5     for (int i = 0; i < L; i++) {
6         sum += X[i];
7     }
8     mean = sum / L;
9     varsum = 0;
10    for (int i = 0; i < L; i++) {
11        varsum += ((X[i] - mean) * (X[i] - mean));
12    }
13    var = varsum / (L - 1);
14    printf("Mean: %f\n", mean);
15    printf("Variance: %f\n", var);
16 }
```

- **第 5 行和第 10 行：**循环语句的边界问题，数组 X 长度为 0、长度为 1、长度大于 2 等情况。
- **数组下界：**考虑数组下界的合理性，如 0 起始点和不越界等问题。
- **潜在约束：**两个循环的次数保持一致。
- **第 11 行：**当 X[i] 跟 mean 两个变量的数值大而且很接近时，容易产生浮点计算的稳定性问题。

讨论与思考： MeanVar 程序中白盒故障假设和黑盒故障假设的相似与区别？

计算稳定性的定义

计算

- 线性代数中的不稳定性：主要关注由于接近各种奇点导致的不稳定性，例如小特征值。
- 微分方程数值算法中的不稳定性：关注舍入误差的增长或初始数据的小波动，可能导致最终结果的较大偏差，例如著名的蝴蝶效应。
- 算法对波动的处理：一些数值算法可能会抑制输入数据中的小波动，而另外一些算法可能会放大波动。

计算稳定性的定义

计算

- 浮点计算与实数运算的差异：浮点计算和实数运算之间的细微差异可能带来严重问题。
- 浮点运算采用不同的舍入模式可能导致不计算结果。
- 例如： $a + (b + c) = (a + b) + c$ 在浮点计算不一定成立。

思考：什么时候会不成立？

- 前向误差：结果与解的差值， $\Delta_y = y' - y$ 。
- 后向误差：最小的 Δ_x 使得 $f(x + \Delta_x) = y'$
- 混合稳定性：结合前向误差和后向误差。

思考：如何进行程序的稳定性测试？

MeanVar程序中方差的计算与稳定性

原始数学公式:
$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1} = \frac{\sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i\right)^2 / n}{n-1}$$

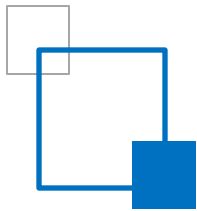
平移不变性: 根据方差的平移不变性, 即对于任意常数 K ,

有 $Var(X - K) = Var(X)$

改进的算法:
$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - K)^2 - \left(\sum_{i=1}^n (x_i - K)\right)^2 / n}{n-1}$$

越接近 K 是平均值, 结果将更准确, 但只需选择样本范围

内的值即可保证所需的稳定性。



软件工程的数值稳定性



选择稳健的数值算法

工程任务是选择稳健的数值算法进行实现，因为计算机采用的是近似计算而不是准确的实数。

计算精度的影响

工程任务侧重考虑计算精度带来的影响，如浮点舍入或截断误差导致与结果偏差被放大等问题。

数值计算的不稳定性

1. 原因：原始数据的精度问题导致的由截断引起的内部错误或输出显著变化。
2. 分类：
 - 数学问题引起的数值不稳定性：难以通过软件技术避免，需要重新定义软件需求。
 - 工程实践引起的数值不稳定性：通过更好的程序设计和编程实践来改进。
3. 应对策略：
 - 检测：检测软件中的数值不稳定性。
 - 分类诊断：对不稳定性问题进行分类诊断。
 - 改进：通过重新定义需求或改进程序设计来解决。

回顾与讨论

黑盒边界
白盒边界



- | |
|---------|
| ➤ 代码视角 |
| ➤ 计算视角 |
| ➤ 单参数边界 |
| ➤ 多参数边界 |
| ➤ 线性边界 |
| ➤ 非线性边界 |



- | |
|----------|
| ➤ 数值计算场景 |
| ➤ 人工智能场景 |